Backend Development – Practical Assignment

Palestinian Automotive Showroom Management System

1. PROJECT BACKGROUND

A Palestinian automotive showroom specializes in selling vehicles from three major brands: **Chery**, **OMODA**, and **JAECOO**. The showroom requires a robust, scalable Backend REST API to manage its vehicle inventory, customer profiles, employee operations, reservations, and sales transactions.

The system supports multi-role access control, accommodating roles such as **Admin**, **Sales Representative**, and **Customer**.

2. TECHNICAL REQUIREMENTS & STACK

Framework: ASP.NET Core Web API (.NET 8.0+)	**ORM:** Entity Framework Core
Database: SQL Server / PostgreSQL	**Auth:** JWT & Role-Based Access Control
Architecture: RESTful API Design	**Scope:** Backend Only (No Frontend)

3. ASSIGNMENT TASK 1 — VEHICLE INVENTORY & TESTING

Objective: Practice Unit Testing and Integration Testing by building and testing a comprehensive vehicle inventory management module.

Part 1: Implement the Vehicle Module

Develop the core vehicle management logic supporting the following capabilities:

- Add a new vehicle to the inventory.
- Update existing vehicle details.
- Delete a vehicle record.
- Retrieve a specific vehicle by ID.
- Retrieve all available vehicles.
- Filter vehicles dynamically by brand (Chery, OMODA, JAECOO) and model.
- Update vehicle availability status.

Field Name	Data Type	Description / Constraints
`Brand`	String / Enum	Chery, OMODA, or JAECOO
`Model`	String	Vehicle model name
`ManufacturingYear`	Integer	Four-digit year (e.g., 2024)
`Price`	Decimal	Price in Palestinian New Shekels (**NIS**)

Field Name	Data Type	Description / Constraints
`Color`	String	Exterior vehicle color
`Mileage`	Integer	Total distance driven in kilometers
`AvailabilityStatus`	Enum	Available, Reserved, or Sold

Part 2: Unit Testing (Business Logic)

Using **xUnit** and **Moq**, write unit tests for the vehicle business logic.

1. Follow the **AAA (Arrange, Act, Assert)** pattern strictly.
2. Use **Mocking** (e.g., Moq) to isolate repository/data access layers.
3. Test the vehicle service independently from database dependencies.
4. Verify successful execution for all standard operational workflows.
5. Validate edge cases and failure scenarios:
   - Adding a vehicle with invalid data (e.g., negative price, missing mandatory fields).
   - Updating a non-existent vehicle ID.
   - Deleting a non-existent vehicle record.
   - Searching for an unavailable or non-existent vehicle.
6. Measure and report test coverage aiming for code coverage standards.

Part 3: Integration Testing (API Pipeline)

Utilize **ASP.NET Core WebApplicationFactory** to conduct integration tests against actual API endpoints.

1. Configure a dedicated test database (e.g., In-Memory DB or Testcontainers).
2. Validate all vehicle CRUD REST endpoints.
3. Verify correct HTTP status codes (`200 OK`, `201 Created`, `400 Bad Request`, `401 Unauthorized`, `403 Forbidden`, `404 Not Found`).
4. Ensure unauthenticated requests to protected endpoints fail properly.
5. Verify role permissions (e.g., ensure Customers cannot add or delete vehicles).

> **Required Deliverables**
>
> - Complete ASP.NET Core Vehicle Management Web API code.
> - Unit testing suite with xUnit & Moq.
> - Integration testing suite with `WebApplicationFactory`.
> - Test coverage summary report.
> - Clear `README.md` with instructions to build, migrate, and run all test suites.